

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №1
по дисциплине «Построение и анализ алгоритмов»
Тема: Поиск с возвратом

Студент гр. 4345

Наумов М. В.

Преподаватель

Жангиров Т. Р.

Санкт-Петербург

2026

Задание

У Вовы много квадратных обрезков доски. Их стороны (размер) изменяются от 1 до $N-1$, и у него есть неограниченное число обрезков любого размера. Но ему очень хочется получить большую столешницу - квадрат размера N . Он может получить ее, собрав из уже имеющихся обрезков(квадратов).

Например, столешница размера 7×7 может быть построена из 9 обрезков.

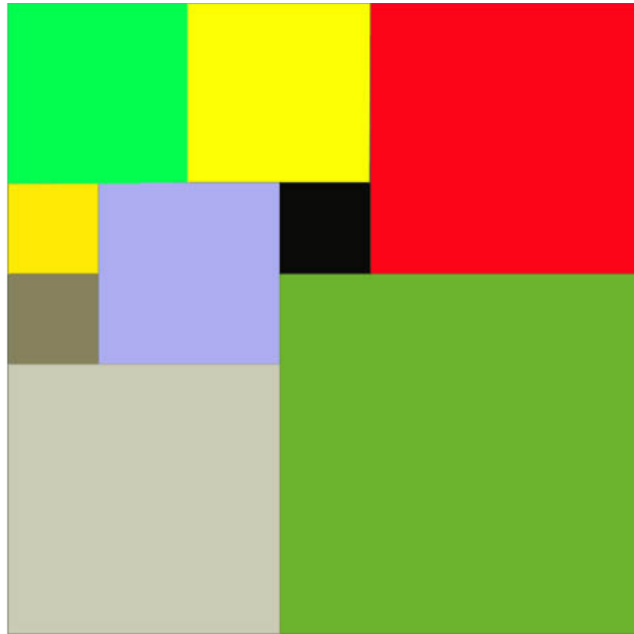


Рисунок 1 – пример расположения квадратов для тестового случая

Внутри столешницы не должно быть пустот, обрезки не должны выходить за пределы столешницы и не должны перекрываться. Кроме того, Вова хочет использовать минимально возможное число обрезков.

Входные данные:

Размер столешницы - одно целое число N ($2 \leq N \leq 40$).

Выходные данные:

Одно число K , задающее минимальное количество обрезков(квадратов), из которых можно построить столешницу(квадрат) заданного размера N . Далее должны идти K строк, каждая из которых должна содержать три целых числа

x, y и w, задающие координаты левого верхнего угла ($1 \leq x, y \leq N$) и длину стороны соответствующего обрезка (квадрата).

Пример входных данных

7

Соответствующие выходные данные

9

1 1 2

1 3 2

3 1 1

4 1 1

3 2 2

5 1 3

4 4 4

1 5 3

3 4 1

Индивидуальный вариант: 1р. Рекурсивный бэктрекинг. Выполнение на Stepik всех трёх заданий в разделе 2.

Описание алгоритма

Основная идея алгоритма заключается в использовании рекурсивного поиска с возвратом (бэктрекинга) с постепенным увеличением глубины, дополненного эвристиками, сокращающими перебор. Программа решает задачу разбиения квадрата $n \times n$ на минимальное число меньших квадратов.

Для чётных n ответ известен (четыре квадрата $n/2$), и алгоритм завершается за $O(1)$. Основная сложность связана с нечётными n . В этом случае сначала выполняется редукция размера: если n составное, задача сводится к меньшему квадрату $d \times d$ (где d — наименьший собственный делитель), так как ответ будет тем же, просто с использованием масштабирования, этот подход уменьшает исходные размеры и значительно сокращает варианты перебора.

Для нечётных n без редукции (или после неё) алгоритм использует эвристическую инициализацию: три квадрата фиксированных размеров ($k, k-1, k-1$, где $k=(n+1)/2$) размещаются в углах, после чего оставшаяся область заполняется рекурсивно. Такой подход уменьшает начальную глубину поиска и очень значительно сокращает пространство состояний.

Далее применяется итеративное углубление: вводится переменная `min_count` (изначально 14), которая служит верхней границей допустимого числа квадратов. Рекурсивная функция `solve` пытается найти решение, используя не более `min_count` квадратов. Если решение не найдено, `min_count` увеличивается на 1, и поиск повторяется. Первое найденное решение гарантированно оптимально, после чего программа завершается.

В рекурсии строго соблюдается детерминированный порядок обхода: на каждом шаге с использованием сохранённой текущей строки отыскивается первая свободная клетка в порядке сканирования слева направо и сверху вниз. Для этой клетки вычисляется максимально возможный размер вписываемого квадрата (с учётом уже занятых клеток), затем перебор размеров выполняется

от максимального к минимальному. Это позволяет быстрее находить решения с большими квадратами и эффективно использовать отсечения.

Для сокращения перебора применяется эвристика по площади: если оставшуюся пустую область невозможно покрыть даже квадратами максимального размера при оставшемся лимите квадратов, ветвь отбрасывается. Это существенно уменьшает количество рассматриваемых вариантов.

Оценка сложности

Временная сложность. В лучшем случае n — чётное число и программе просто достаточно вывести известное разбиение на 4 квадрата. В таком случае сложность алгоритма равна $O(1)$.

В худшем случае определяется глубиной рекурсии L (текущий лимит квадратов) и размером стороны n . На каждом уровне рекурсии:

- Поиск первой пустой клетки требует $O(n^2)$ операций.
- Определение максимального размера квадрата и последующий перебор всех возможных размеров требуют до $O(n^2)$ операций.
- Обновление матрицы при установке квадрата занимает $O(n^2)$.

Таким образом, анализ каждой клетки в худшем случае занимает имеет сложность $O(n^2)$. В обычном случае количество сочетаний размеров бы до $O(n^n)$ и алгоритм имел бы показательную сложность, которая является колоссальной. Но поскольку мы ввели `min_count`, который равен отдельному значению и меняется, то сложность равна $O(n^2) * O(n^{\text{min_count}}) = O(n^{2+\text{min_count}})$

Сложность по памяти определяется:

- Матрицей состояния `table[45][45]`, занимающей $O(n^2)$ (лишние клетки были выделены для подстраховки и могут быть убраны).

- Глубиной рекурсии, которая не превышает текущего лимита L (максимум n^2).
- Вектором текущего решения, хранящим не более `min_count` квадратов.

Таким образом, общая память составляет $O(n^2 + \text{min_count})$, что вполне приемлемо для $n \leq 40$.

Оптимизация алгоритма

Несмотря на высокую оценку сложности в худшем случае, программа использует две ключевые оптимизации, которые на практике сокращают перебор до приемлемого времени даже для максимального размера $n=40$.

1. Редукция размера для составных n

Если n имеет делитель d (большой 1), то исходный квадрат можно представить как $n=d \cdot k$. В этом случае всякое разбиение квадрата $d \times d$ на меньшие квадраты после масштабирования всех сторон в k раз даёт допустимое разбиение квадрата $n \times n$. Поэтому программа находит наименьший делитель d и рекурсивно решает задачу для квадрата $d \times d$, после чего просто умножает координаты и размеры полученных квадратов на k .

2. Фиксированное начальное размещение для нечётных n

Для нечётных n (в том числе для уменьшенного размера d после редукции) алгоритм использует геометрическую эвристику: в левый верхний угол ставится квадрат со стороной $(n+1)/2$, а справа от него и снизу — два квадрата со стороной $n/2$. Эти три квадрата покрывают чуть более 75% площади, оставляя незаполненной лишь небольшую область в правом нижнем углу. Такая принудительная расстановка резко сокращает дерево поиска: дальнейший перебор ведётся только по оставшейся области, которая имеет малую площадь и специфическую форму. В результате бэктрекинг быстро находит оптимальное заполнение этой области, используя оставшийся лимит квадратов.

3. Эвристика по площади (отсечение по остатку)

На каждом шаге рекурсии, перед тем как попробовать очередной размер квадрата, алгоритм оценивает, можно ли в принципе заполнить оставшуюся пустую область, используя не более оставшегося количества квадратов (текущий лимит `min_count` минус уже использованные `count`). Для этого вычисляется максимальная площадь, которую можно покрыть, если каждый следующий квадрат будет иметь наибольший возможный размер — сторона оставшейся области ограничена текущей позицией, но для грубой оценки берётся максимальный размер, равный `n - start_y` (высота оставшейся части) и находится значение $(n - start_y) * (n - start_y) * (min_count - count - 1)$. Умножение на оставшееся количество квадратов даёт верхнюю оценку площади, которую ещё можно заполнить. Если эта оценка меньше оставшейся пустой площади `empty - w * w`, то даже при самом оптимистичном раскладе заполнить оставшуюся область не удастся, и дальнейший перебор для этого размера и всех меньших (так как они дадут ещё меньшую заполненную площадь) бесполезен. В этом случае функция возвращается, отсекая целую ветвь дерева поиска. Эта эвристика особенно эффективна на поздних этапах, когда область мала, и позволяет отбросить множество заведомо неперспективных комбинаций.

Способ хранения частичных решений

Для представления частичных решений в процессе поиска не используется копирование доски на каждом шаге — это потребовало бы значительных затрат памяти. Вместо этого применяется единый глобальный массив `board` (размером 45×45), где значение 1 означает занятую клетку, а 0 — свободную. Параллельно ведётся вектор `current_solution`, который функционирует как стек: в нём сохраняются параметры уже размещённых квадратов (координаты верхнего левого угла и сторона).

При добавлении нового квадрата соответствующие ячейки массива помечаются единицами, а информация о квадрате записывается в конец вектора.

В случае необходимости отката (при возврате из рекурсии) последний элемент вектора удаляется, а занятые им клетки вновь обнуляются. Если найдено полное заполнение доски, содержимое `current_solution` копируется в вектор `best_res`, хранящий наилучшее на данный момент решение. Такой подход обеспечивает эффективное использование памяти и быстрый доступ к текущему состоянию.

Описание рекурсивной функции (бэктрекинг)

Рекурсивная функция `solve` является ядром алгоритма поиска с возвратом. Она принимает два параметра: текущее количество уже использованных квадратов (`count`) и оставшуюся пустую площадь (`empty`). Её задача — заполнить всю доску, не превышая установленного лимита квадратов (`min_count`).

На каждом шаге функция сначала сканирует доску в строго определённом порядке: слева направо и сверху вниз. Как только встречается свободная клетка (обозначенная нулём), её координаты запоминаются как `start_x` и `start_y`. Если свободных клеток не найдено, значит доска полностью заполнена — функция возвращает `true`, сигнализируя об успешном завершении.

Для найденной позиции вычисляется максимально возможный размер квадрата, который можно вписать. Этот размер ограничен расстоянием до правого края и нижнего края доски, а также необходимостью не накладываться на уже занятые клетки. Затем происходит перебор всех возможных размеров от этого максимума вниз до единицы.

Для каждого пробного размера `w` сначала проверяется, можно ли разместить квадрат (все клетки в соответствующей области свободны). Если проверка пройдена, то эти клетки помечаются как занятые (в них записывается единица), а информация о квадрате добавляется в конец вектора

current_solution. Затем рекурсивно вызывается solve с параметрами count + 1 и empty - w*w.

Если рекурсивный вызов возвращает true, значит решение найдено, и функция немедленно передаёт true на верхний уровень. В противном случае выполняется откат: занятые клетки снова обнуляются, а последний элемент удаляется из вектора current_solution. После этого цикл переходит к следующему (меньшему) размеру.

Если ни один из размеров не привёл к успеху, функция возвращает false, что означает необходимость возврата на предыдущий уровень рекурсии для перебора других вариантов.

Перед рекурсивным вызовом также применяется эвристика по площади: если даже при использовании максимально возможных квадратов на оставшихся шагах нельзя покрыть всю оставшуюся пустую площадь, то текущий размер отбрасывается без погружения в рекурсию. Это позволяет сократить перебор и отсечь заведомо бесперспективные ветви.

Описание функций и структур данных

****Структуры данных и глобальные переменные****

- struct Square — модель одного размещённого квадрата. Содержит координаты левого верхнего угла (x, y) и длину стороны (w).

- int table[45][45] — глобальная матрица, представляющая текущее заполнение доски. Значение 0 означает свободную клетку, 1 — занятую. Используется один общий массив, что исключает копирование состояния при рекурсивных вызовах.

- int n — глобальный размер текущей доски (после возможного масштабирования).

- `int min_count` — текущий лимит на количество квадратов (аналог `LIMIT`). Изначально равен 14 и может увеличиваться, если решение не найдено.
- `bool found` — флаг, сигнализирующий, что оптимальное решение уже обнаружено и дальнейший поиск можно прекратить.
- `std::vector<Square> best_res` — вектор, в который сохраняется лучшее (первое найденное) решение.

****Рекурсивная функция solve****

Сигнатура:

```
void solve(int count, std::vector<Square>& squares, int min_y, int empty)
```

Параметры:

- `count` — количество уже использованных квадратов (включая три предустановленных).
- `squares` — вектор, хранящий текущий набор квадратов (работает как стек).
- `min_y` — оптимизация для ускорения поиска: сканирование свободных клеток начинается со строки `min_y`, а не с нуля.
- `empty` — оставшаяся пустая площадь (обновляется при добавлении квадрата).

Логика работы:

1. ****Проверка лимита и поиск свободной клетки.****

Если `count >= min_count`, дальнейший поиск в этой ветви прекращается (возврат).

Иначе выполняется сканирование доски, начиная со строки `min_y`, слева направо и сверху вниз, до первой клетки со значением 0. Её координаты

запоминаются как (start_x, start_y). Если свободных клеток нет — доска полностью заполнена. Тогда:

- если $\text{count} < \text{min_count}$, обновляется min_count (становится равным count), флаг found устанавливается в true, а текущий вектор squares копируется в best_res;
- функция возвращается.

2. **Определение максимального возможного размера квадрата.**

Для позиции (start_y, start_x) последовательно проверяется, можно ли вписать квадрат со стороной, увеличивающейся от 1 до предела, определяемого границами доски (n) и занятыми клетками. Проверка выполняется по правому и нижнему краям пробного квадрата. Наибольший успешно проверенный размер записывается в max_w.

3. **Эвристика по площади (отсечение).**

Перед перебором размеров вычисляется условие:

$$(n - \text{start_y}) * (n - \text{start_y}) * (\text{min_count} - \text{count} - 1) < \text{empty} - w * w.$$

Если оно выполняется для текущего w, то даже при заполнении оставшейся области максимально возможными квадратами (со стороной не более $n - \text{start_y}$) нельзя покрыть оставшуюся пустую площадь. В этом случае дальнейший перебор для данного и меньших размеров прекращается (функция возвращается).

4. **Перебор размеров от max_w до 1.**

Для каждого размера w:

- Проверяется, можно ли разместить квадрат (все клетки в области (start_y ... start_y+w-1, start_x ... start_x+w-1) свободны). Если нет — проверка для меньших размеров продолжается, так как они могут поместиться (они находятся внутри той же области и могут не затрагивать занятые клетки).
- Если квадрат можно поставить, то:

- * соответствующие ячейки table заполняются единицами;
- * квадрат добавляется в конец вектора squares;
- * рекурсивно вызывается solve(count + 1, squares, start_y, empty - w*w).
- После возврата из рекурсии:
 - * если found истинно и count + 1 не превышает новый min_count (условие для раннего выхода), функция немедленно возвращается;
 - * иначе выполняется откат: занятые клетки снова обнуляются, последний элемент удаляется из squares.
- Переход к следующему (меньшему) размеру.

5. Если ни один размер не привёл к успеху, функция просто завершается (возврат на предыдущий уровень).

****Главная функция main****

1. Считывает исходный размер original_n.
2. Если original_n чётное, сразу формируется решение из четырёх квадратов со стороной original_n/2, выводится и программа завершается.
3. Для нечётного original_n:
 - Проверяется, есть ли у него делитель d (кроме 1). Если да, то задача масштабируется: $n = d$, $scale = original_n / d$. Это позволяет решать задачу для меньшего квадрата.
 - Вычисляются базовые размеры для эвристической расстановки:
 $k = (n + 1) / 2$ (квадрат в левом верхнем углу),
 два квадрата со стороной k-1 размещаются справа от первого и под ним (в углах).
 - Доска (table) заполняется этими тремя квадратами, вычисляется их суммарная площадь filled.
 - Вектор initial_squares инициализируется этими тремя квадратами.
 - Запускается цикл while (!found):

* вызывается solve(3, initial_squares, 0, empty_space) (где empty_space = n*n - filled);

* если решение не найдено (found == false), лимит min_count увеличивается на 1 и поиск повторяется.

- Как только found становится истинным, цикл завершается. Найденное в best_res решение содержит минимальное число квадратов для текущего n.

4. Если ранее было применено масштабирование, координаты и размеры всех квадратов в best_res умножаются на коэффициент scale (с учётом того, что в выводе координаты должны начинаться с 1).

5. Выводится размер best_res и затем каждый квадрат в формате: x y w.

Тестирование

Результаты тестирования представлены в табл. 1.

Таблица 1 – Результаты тестирования

№ п/п	Входные данные	Выходные данные	Комментарии
1.	7	9 1 1 4 5 1 3 1 5 3 5 4 2 7 4 1 4 5 1 7 5 1 4 6 2 6 6 2	Простое поведение программы
2.	39	6 1 1 26 27 1 13 1 27 13 27 14 13 14 27 13	Программа считает верно и крайне быстро, так как делит квадрат и масштабирует

		27 27 13	
3.	40	4 1 1 20 1 21 20 21 1 20 21 21 20	В случае с чётными числами всё просто и мгновенно
4.	37	15 1 1 19 20 1 18 1 20 18 20 19 2 22 19 5 27 19 11 19 20 1 19 21 3 19 24 8 27 30 3 30 30 8 19 32 6 25 32 1 26 32 1 25 33 5	Большие простые числа не заставляют программу долго “думать”, и она выдаёт верный ответ.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Название файла: squares.cpp

```
#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;

struct Square {
    int x, y, w;
};

int n;
int min_count = 14;
bool found = false;
vector<Square> best_res;
int table[45][45];

void solve(int count, vector<Square>& squares, int min_y, int empty) {
    if (count >= min_count) return;

    int start_y = -1, start_x = -1;
    for (int r = min_y; r < n; ++r) {
        for (int c = 0; c < n; ++c) {
            if (table[r][c] == 0) {
                start_y = r;
                start_x = c;
                break;
            }
        }
        if (start_y != -1) break;
    }

    if (start_y == -1) {
        if (count < min_count) {
            min_count = count;
            found = true;
            best_res = squares;
        }
        return;
    }
}
```

```

    }

    int max_w = 0;
    int limit = min(n - start_y, n - start_x);

    for (int w = 1; w <= limit; ++w) {
        bool possible = true;
        for (int i = 0; i < w; ++i) {
            if (table[start_y + i][start_x + w - 1] == 1 ||
                table[start_y + w - 1][start_x + i] == 1) {
                possible = false;
                break;
            }
        }
        if (!possible) break;
        max_w = w;
    }

    for (int w = max_w; w >= 1; --w) {
        if ((n - start_y) * (n - start_x) * (min_count - count - 1) < empty -
            w * w) {
            return;
        }

        for (int r = start_y; r < start_y + w; ++r)
            for (int c = start_x; c < start_x + w; ++c)
                table[r][c] = 1;

        squares.push_back({start_x + 1, start_y + 1, w});

        solve(count + 1, squares, start_y, empty - w * w);

        squares.pop_back();
        for (int r = start_y; r < start_y + w; ++r)
            for (int c = start_x; c < start_x + w; ++c)
                table[r][c] = 0;

        if (found && min_count <= count + 1) return;
    }
}

int main() {
    ios::sync_with_stdio(false);

```

```

cin.tie(NULL);

if (!(cin >> n)) return 0;

for(int i = 0; i < 45; i++)
    for(int j = 0; j < 45; j++) table[i][j] = 0;

int scale = 1;
int original_n = n;
int new_n = 0;

if (n % 2 != 0) {
    for (int d = 2; d <= n / 2; ++d) {
        if (n % d == 0) {
            new_n = d;
            scale = n / d;
            break;
        }
    }
    if (new_n != 0) n = new_n;

    int k = (n + 1) / 2;
    for (int r = 0; r < k; ++r)
        for (int c = 0; c < k; ++c) table[r][c] = 1;
    for (int r = 0; r < k - 1; ++r)
        for (int c = k; c < n; ++c) table[r][c] = 1;
    for (int r = k; r < n; ++r)
        for (int c = 0; c < k - 1; ++c) table[r][c] = 1;

    vector<Square> initial_squares = {
        {1, 1, k},
        {k + 1, 1, k - 1},
        {1, k + 1, k - 1}
    };

    int empty_space = n * n - (k * k + 2 * (k - 1) * (k - 1));

    while (!found) {
        solve(3, initial_squares, 0, empty_space);
        if (!found) min_count++;
    }

    if (new_n != 0) {

```

```

        for (int i = 0; i < (int)best_res.size(); ++i) {
            best_res[i].x = (best_res[i].x - 1) * scale + 1;
            best_res[i].y = (best_res[i].y - 1) * scale + 1;
            best_res[i].w *= scale;
        }
    }
} else {
    int half = n / 2;
    best_res = {
        {1, 1, half},
        {1, half + 1, half},
        {half + 1, 1, half},
        {half + 1, half + 1, half}
    };
    min_count = 4;
}

cout << best_res.size() << "\n";
for (int i = 0; i < (int)best_res.size(); ++i) {
    cout << best_res[i].x << " " << best_res[i].y << " " << best_res[i].w
<< "\n";
}

return 0;
}

```